Prelucrarea Imaginilor

Prof. Dr. Călin-Adrian POPA

Cursul 8

17 Noiembrie 2025

8 Image captioning

- *image captioning* represents the task of describing the content of an image in natural language, i.e., generating captions for an image, thus its name
- it is a fundamental problem in artificial intelligence that connects computer vision and natural language processing
- it mimicks the remarkable human ability to compress huge amounts of visual infomation into descriptive language

8 Image captioning



A female tennis player in action on the court.



A group of young men playing a game of soccer



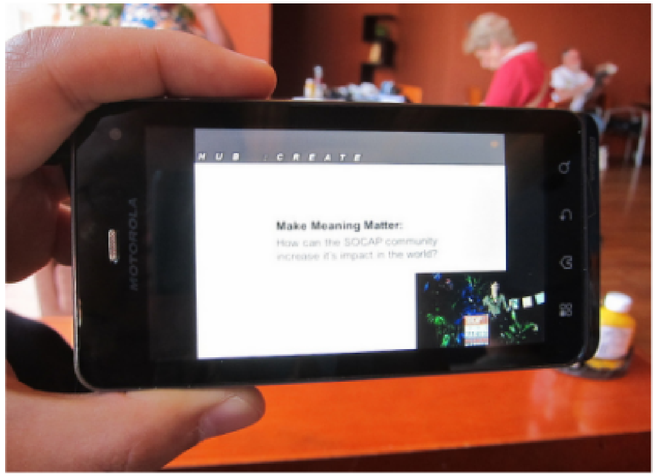
A man riding a wave on top of a surfboard.



A baseball game in progress with the batter up to plate.



A brown bear standing on top of a lush green field.



A person holding a cell phone in their hand.

8 Image captioning

- the task of free-form and open-ended *visual question answering (VQA)*: given an image and a natural language question about the image, the task is to provide an accurate natural language answer
- mirroring real-world scenarios, such as helping the visually impaired, both the questions and answers are open-ended
- visual questions selectively target different areas of an image, including background details and underlying context
- as a result, a system that succeeds at VQA typically needs a more detailed understanding of the image and complex reasoning than a system producing generic image captions
- nonetheless, many open-ended answers contain only a few words or a closed set of answers that can be provided in a multiple-choice format

8 Image captioning



Q: What endangered animal is featured on the truck?

A: **A bald eagle.**
A: A sparrow.
A: A humming bird.
A: A raven.



Q: Where will the driver go if turning right?

A: **Onto 24 ¾ Rd.**
A: Onto 25 ¾ Rd.
A: Onto 23 ¾ Rd.
A: Onto Main Street.



Q: When was the picture taken?

A: **During a wedding.**
A: During a bar mitzvah.
A: During a funeral.
A: During a Sunday church service.



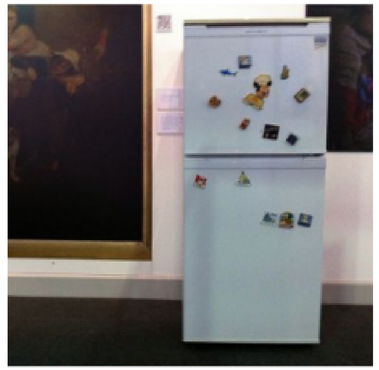
Q: Who is under the umbrella?

A: **Two women.**
A: A child.
A: An old man.
A: A husband and a wife.



Q: Why was the hand of the woman over the left shoulder of the man?

A: **They were together and engaging in affection.**
A: The woman was trying to get the man's attention.
A: The woman was trying to scare the man.
A: The woman was holding on to the man for balance.



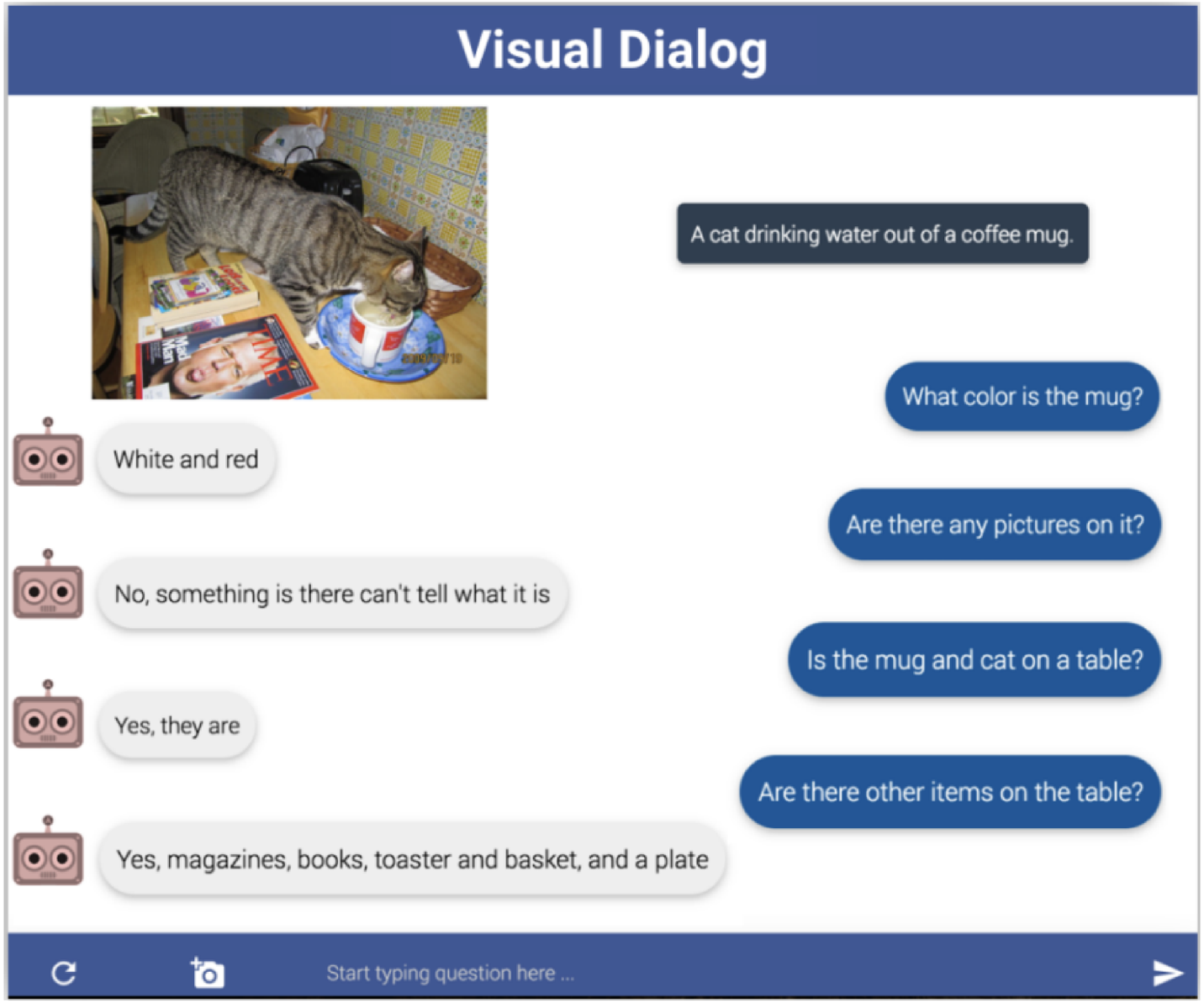
Q: How many magnets are on the bottom of the fridge?

A: **5.**
A: 2.
A: 3.
A: 4.

8 Image captioning

- the task of *visual dialog* requires an agent to hold a meaningful dialog with humans in natural, conversational language about visual content
- specifically, given an image, a dialog history, and a question about the image, the agent has to ground the question in the image, infer context from history, and answer the question accurately
- visual dialog is disentangled enough from a specific downstream task so as to serve as a general test of machine intelligence, while being grounded in vision enough to allow objective evaluation of individual responses and benchmark progress

8 Image captioning



8.1 Show and Tell

- Neural Image Caption (NIC) represents a single joint model that takes an image I as input, and is trained to maximize the likelihood $p(S|I)$ of producing a target sequence of words $S = \{S_1, S_2, \ldots\}$, where each word S_t comes from a given dictionary, that describes the image adequately
- the main inspiration comes from machine translation, where the task is to transform a sentence S written in a source language, into its translation T in the target language, by maximizing $p(T|S)$
- this problem is solved by using RNNs
- an encoder RNN reads the source sentence and transforms it into a rich fixed-length vector representation, which in turn is used as the initial hidden state of a decoder RNN that generates the target sentence

8.1 Show and Tell

- NIC follows this recipe, replacing the encoder RNN by a deep CNN
- CNNs produce a rich representation of the input image by embedding it to a fixed-length vector, such that this representation can be used for a variety of vision tasks
- hence, it is natural to use a CNN as an image encoder, by first pre-training it for an image classification task and using the last hidden layer as an input to the RNN decoder that generates sentences (see Figure 1)
- NIC is an end-to-end trainable neural network for image captioning
- the model combines sub-networks for vision and language models that can be pre-trained on larger corpora and thus can take advantage of additional data

8.1 Show and Tell

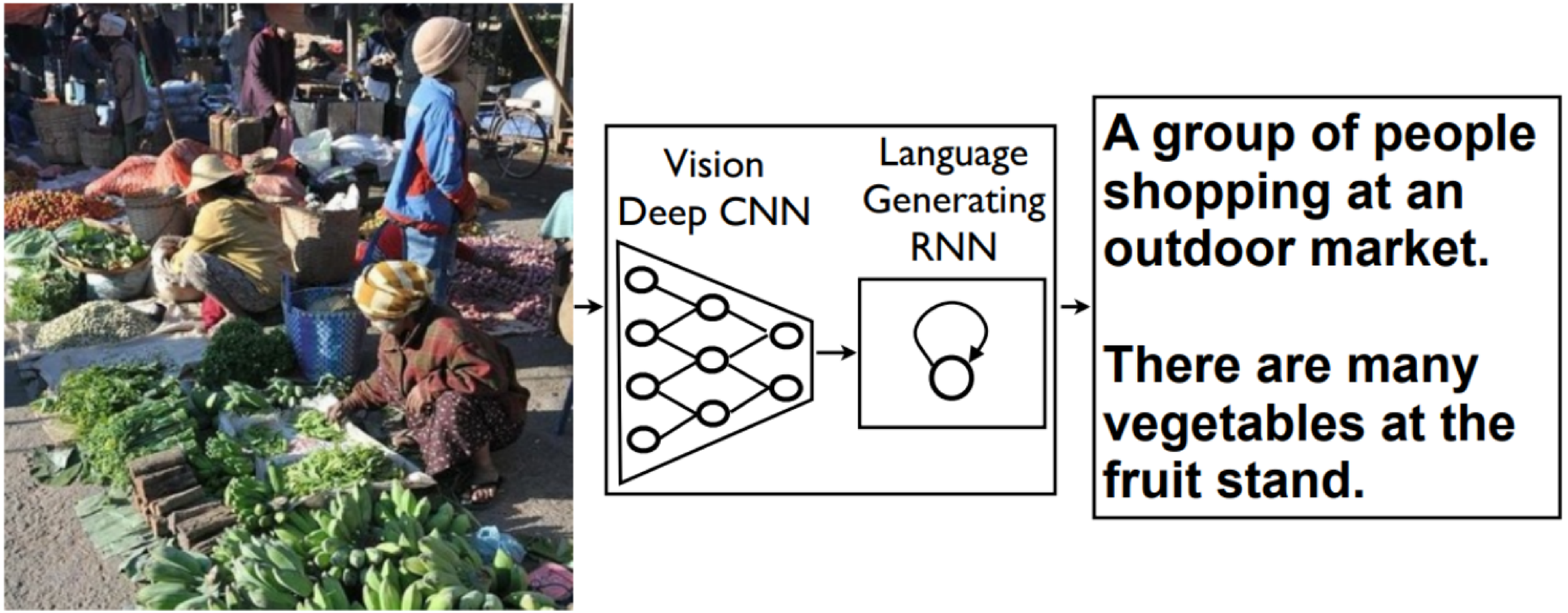


Figure 1: The Neural Image Caption (NIC) model is based end-to-end on a neural network consisting of a vision CNN followed by a language generating RNN. It generates complete sentences in natural language from an input image, as shown in the example.

8.1 Show and Tell

- its purpose is to directly maximize the probability $p(S|I)$ of the correct description S given the image I
- since S represents any sentence, its length is unbounded
- thus, it is common to model the joint probability over $S_0, \ldots, S_N$, where N is the length of the description as:

$$p(S|I) = \prod_{t=0}^{N} p(S_t|I, S_0, \ldots, S_{t-1}).$$

- it is natural to model $p(S_t|I, S_0, \ldots, S_{t-1})$ with an RNN, and, for the NIC model, the LSTM network is used
- the LSTM model is trained to predict each word of the sentence after it has seen the image, as well as all preceding words, as defined by $p(S_t|I, S_0, \ldots, S_{t-1})$
- for this purpose, it is instructive to think of the LSTM in unrolled form – a copy of the LSTM memory is created for the image and each sentence word, such that all LSTMs share the same parameters and the output of the LSTM at time $t-1$ is fed to the LSTM at time t (see Figure 2)

8.1 Show and Tell

- all recurrent connections are transformed to feed-forward connections in the unrolled version
- in more detail, if we denote by I the input image and by $S = (S_0, \ldots, S_N)$ a true sentence describing this image, the unrolling procedure is as follows:

$$\begin{aligned} x_{-1} &= CNN(I) \\ x_t &= W_e S_t, \quad t \in \{0, \ldots, N-1\} \\ p_{t+1} &= LSTM(x_t), \quad t \in \{0, \ldots, N-1\} \end{aligned}$$

- here, we represent each word as a one-hot vector S_t of dimension equal to the size of the dictionary
- note that we denote by S_0 a special start word and by S_N a special stop word, which designate the start and end of the sentence, respectively
- in particular, by emitting the stop word, the LSTM signals that a complete sentence has been generated
- both the image and the words are mapped to the same space: the image by using a vision CNN, the words by using word embedding matrix W_e

8.1 Show and Tell

- the image I is only input once, at $t = -1$, to inform the LSTM about the image contents
- it was empirically verified that feeding the image at each time step as an extra input yields inferior results, as the network can explicitly exploit noise in the image and overfits more easily
- the loss is the sum of the negative log-likelihood of the correct word at each step as follows:

$$L(I, S) = -\sum_{t=1}^{N} \log p_t(S_t).$$

- the above loss is minimized with respect to all the parameters of the LSTM, the top layer of the image embedder CNN, and word embeddings W_e
- all weights were randomly initialized except for the CNN weights, which were pretrained on ImageNet
- the dimension for the embeddings and the size of the LSTM memory were set to 512

8.1 Show and Tell

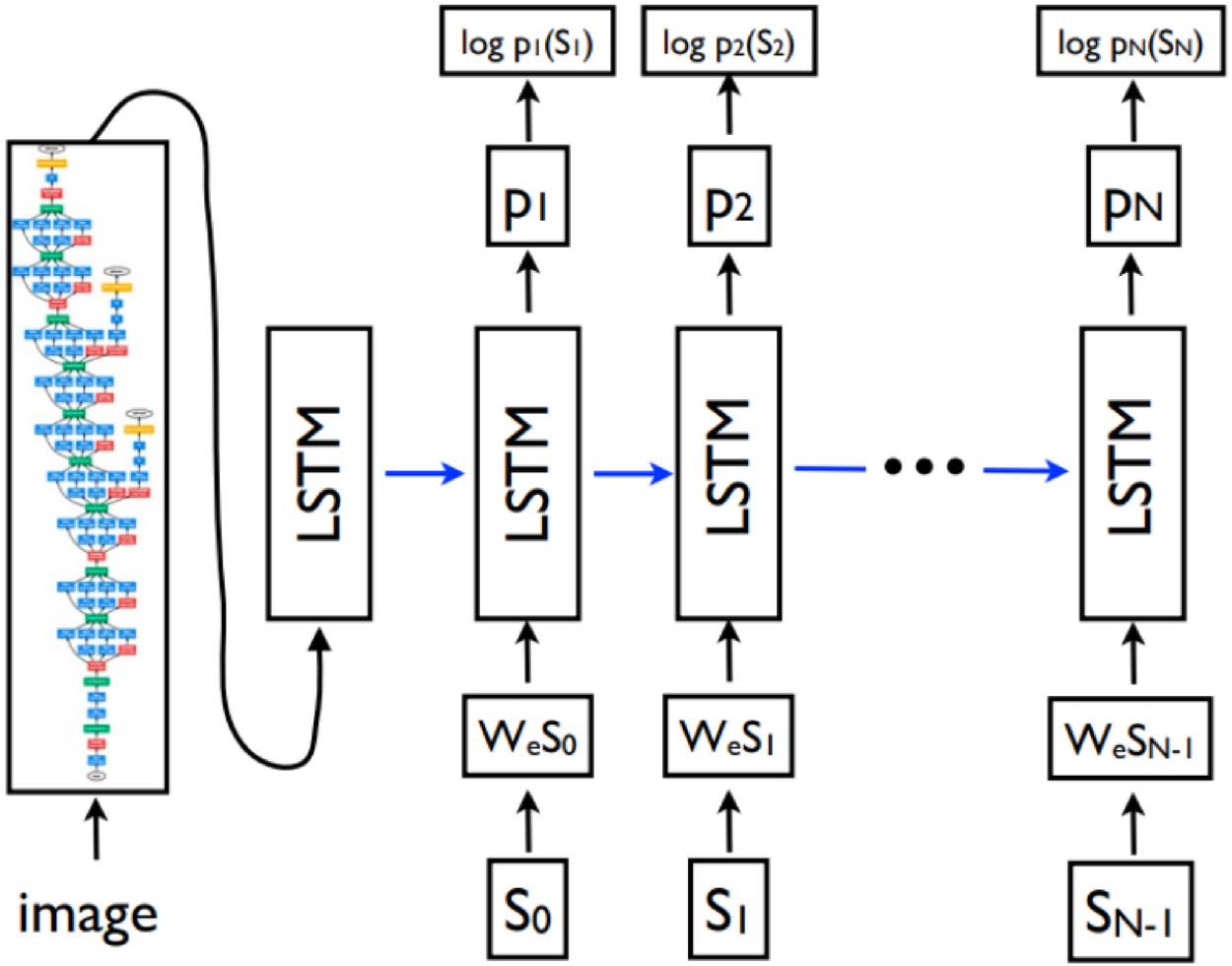


Figure 2: LSTM model combined with a CNN image embedder and word embeddings. The unrolled connections between the LSTM memories are in blue and they correspond to the recurrent connections in the LSTMs. All LSTMs share the same parameters.

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

- ideally, an image captioning model should provide variable length outputs, including generation of full-length sentence descriptions that go beyond conventional one-versus-all prediction tasks
- Long-term Recurrent Convolutional Networks (LRCNs) represent a class of architectures for visual recognition and description which combine convolutional layers and long-range temporal recursion and are end-to-end trainable (Figure 3)
- it can be instantiated for specific video activity recognition, image caption generation, and video description tasks
- LRCNs show that convolutional networks with recurrent units are generally applicable to visual time-series modeling
- in visual tasks, LSTM-style RNNs can provide significant improvement when ample training data are available to learn or refine the representation
- specifically, LSTM-type models enable a novel end-to-end optimizable mapping from image pixels to sentence-level natural language descriptions

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

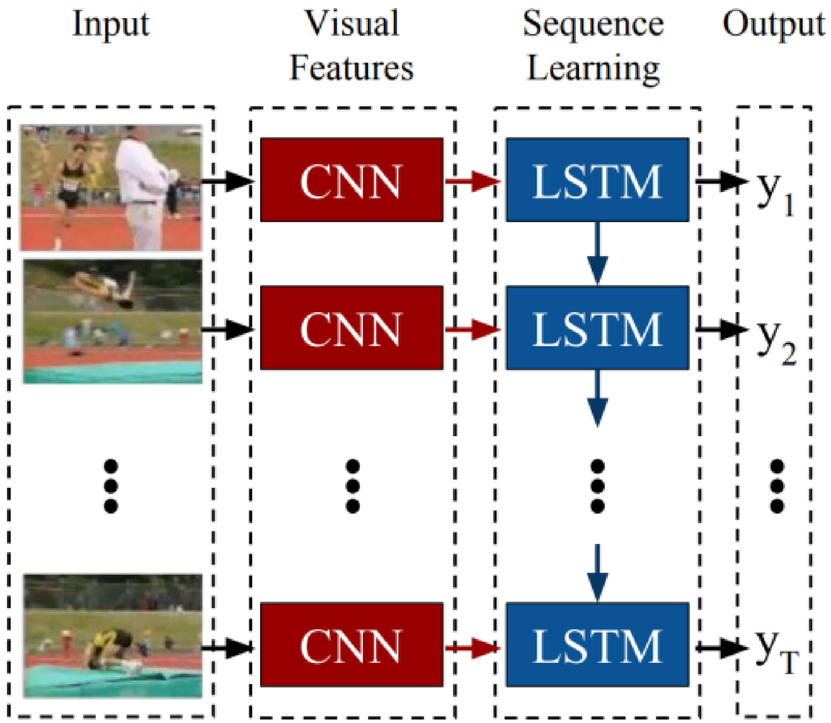


Figure 3: Long-term Recurrent Convolutional Networks (LRCNs), a class of architectures leveraging the strengths of rapid progress in CNNs for visual recognition problems, and the growing desire to apply such models to time-varying inputs and outputs. LRCN processes the (possibly) variable-length visual input (left) with a CNN (middle-left), whose outputs are fed into a stack of recurrent sequence models (LSTMs, middle-right), which finally produce a variable-length prediction (right). Both the CNN and LSTM weights are shared across time, resulting in a representation that scales to arbitrarily long sequences.

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

- LRCNs construct end-to-end trainable image to sentence mappings
- machine translation models are encoder-decoder pairs based on LSTM networks
- LRCNs are a multimodal analog of this model, and their architecture uses a visual CNN to encode a deep state vector, and an LSTM to decode the vector into a natural language string (Figure 4)
- the resulting model can be trained end-to-end on large-scale image and text datasets

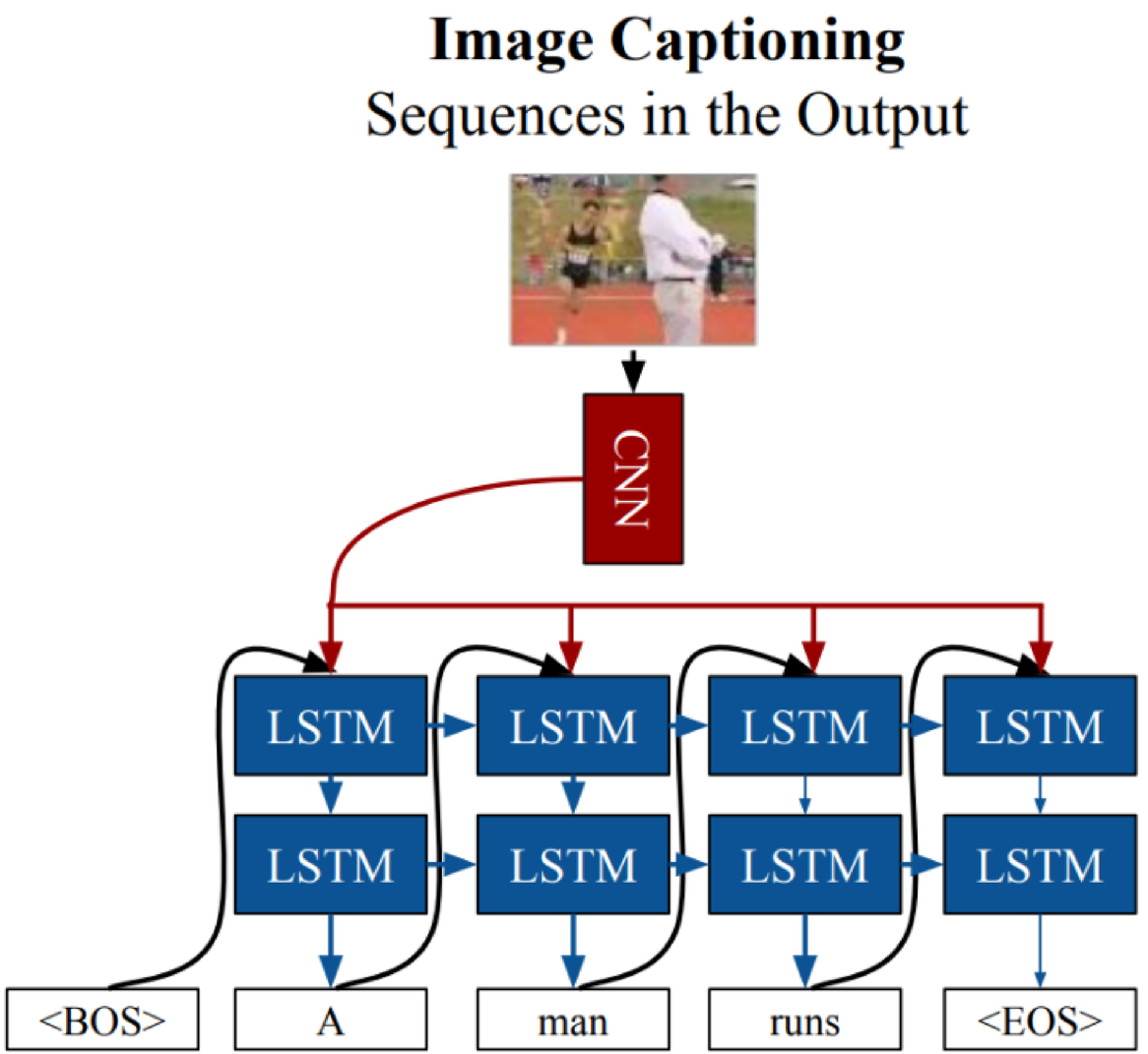


Figure 4: Task-specific instantiation of the LRCN model for image captioning.

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

- LRCN works by passing the visual input x (an image) through a feature transformation $\phi_V(\cdot)$ with parameters V, usually the activations in some layer of a deep CNN, to produce a fixed-length vector representation $\phi_V(x)$
- the outputs of ϕ_V are then passed into a recurrent sequence learning module
- some models stack multiple LSTMs on top of each other, as described later
- to predict a distribution $P(y_t)$ over outcomes $y_t \in C$ (where C is a discrete, finite set of outcomes) at time step t, the outputs of the sequential model are passed through a linear prediction layer
- finally, the predicted distribution $P(y_t)$ is computed by taking the softmax of the linear prediction layer
- the image captioning problem represents the mapping $x \mapsto \langle y_1, y_2, \ldots, y_T \rangle$, with a static image as input, but a much larger and richer label space consisting of sentences of any length

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

- in order to perform image captioning, at each time step, both the image features and the previous word are provided as inputs to the sequence model, in this case a stack of LSTMs (each with 1000 hidden units), which is used to learn the dynamics of the time-varying output natural language sequence
- at time step t, the input to the bottom-most LSTM in Figure 4 is the embedded word from the previous time step y_{t-1}
- input words are encoded as "one-hot" vectors: vectors $y \in \mathbb{R}^K$ with a single non-zero component $y_i = 1$ denoting the ith word in the vocabulary, where K is the number of words in the vocabulary, plus one additional entry for the <BOS> (beginning of sequence) token, which is always taken as y_0, the "previous word" at the first time step ($t = 1$)
- these one-hot vectors are then projected into an embedding space with dimension d_e by multiplication $W_e y_t$ with a learned parameter matrix $W_e \in \mathbb{R}^{d_e \times K}$
- the result of a matrix-vector multiplication with a one-hot vector is the column of the matrix corresponding to the index of the single nonzero component of the one-hot vector
- W_e can therefore be thought of as a "lookup table", mapping each of the K words in the vocabulary to a d_e-dimensional vector

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

- the visual feature representation $\phi_V(x)$ of the image x may be input to the sequence model – a stack of L LSTMs – by concatenating it at each time step either with (1) the embedded previous word $W_e y_{t-1}$ and fed into the first LSTM of the stack, or (2) the hidden state $h_t^{(\ell-1)}$ output from LSTM $\ell - 1$ and fed into LSTM ℓ, for some $\ell \in \{2, \ldots, L\}$
- these choices are depicted in Figure 5
- we refer to the latter choice as "factored", as it forces a sort of separation of responsibilities by "blinding" the first $\ell - 1$ LSTMs and forcing all of the capacity of their hidden states at time step t to represent only the partial caption $y_{1:t-1}$ independent of the visual input, while the LSTMs starting from ℓ are responsible for fusing the lower layer's hidden state given by the partial caption with the visual feature representation $\phi_V(x)$ to produce a joint hidden state representation $h_t^{(\ell)}$ of the visual and language inputs up to time step t from which the next word y_t can be predicted
- in the factored case, the hidden state h_t for the lower layers is conditionally independent of the image x given the partial caption $y_{1:t-1}$

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

- the outputs of the final LSTM in the stack are the inputs to a learned linear prediction layer with a softmax producing a distribution $P(y_t|y_{1:t-1}, \phi_V(x))$ over words y_t in the model's vocabulary, including the <EOS> token denoting the end of the caption, allowing the model to predict captions of varying length
- the visual model ϕ_V used is the VGG model pre-trained for ImageNet classification
- without any explicit language modeling or impositions on the structure of the generated captions, the described LRCN system learns mappings from images input as pixel intensity values to natural language descriptions that are often semantically descriptive and grammatically correct
- at training time, the previous word inputs $y_{1:t-1}$ at time step t are from the ground truth caption
- for inference of captions on a novel image x, the input is a sample $y_t \sim P(y_t|y_{1:t-1}, \phi_V(x))$ from the model's predicted distribution at the previous time step, and generation continues until an <EOS> (end of sequence) token is generated

8.2 Long-term Recurrent Convolutional Networks (LRCNs)

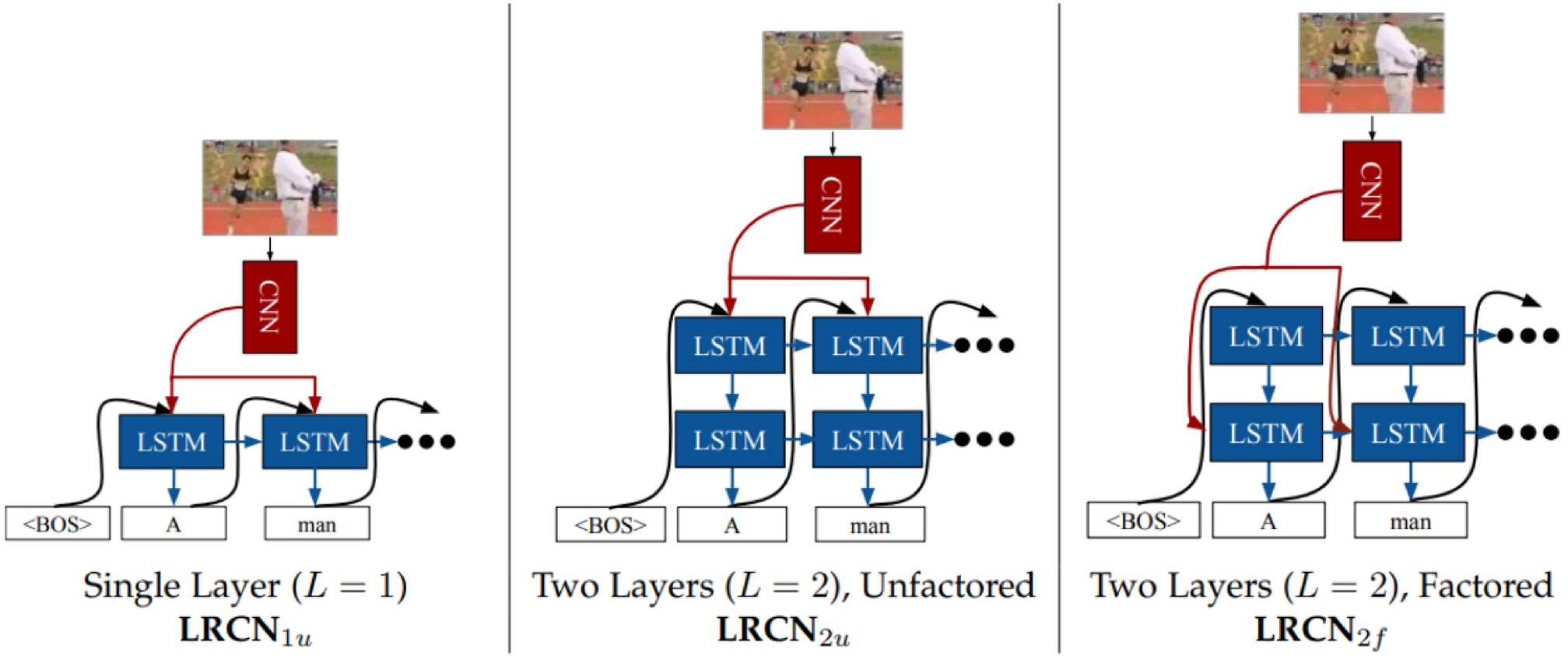


Figure 5: Three variants of the LRCN image captioning architecture. It is explored the effect of depth in the LSTM stack, and the effect of the "factorization" of the modalities.

8.3 Deep Visual-Semantic Alignments

- this work aims to take a step towards the goal of generating dense descriptions of images (Figure 6)
- the primary challenge towards this goal is in the design of a model that is rich enough to simultaneously reason about contents of images and their representation in the domain of natural language
- additionally, the model should be free of assumptions about specific hard-coded templates, rules, or categories and instead rely on learning from the training data
- the second, practical challenge is that datasets of image captions are available in large quantities, but these descriptions multiplex mentions of several entities whose locations in the images are unknown
- the core insight is that we can leverage these large image-sentence datasets by treating the sentences as weak labels, in which contiguous segments of words correspond to some particular, but unknown location in the image
- the approach is to infer these alignments and use them to learn a generative model of descriptions

8.3 Deep Visual-Semantic Alignments

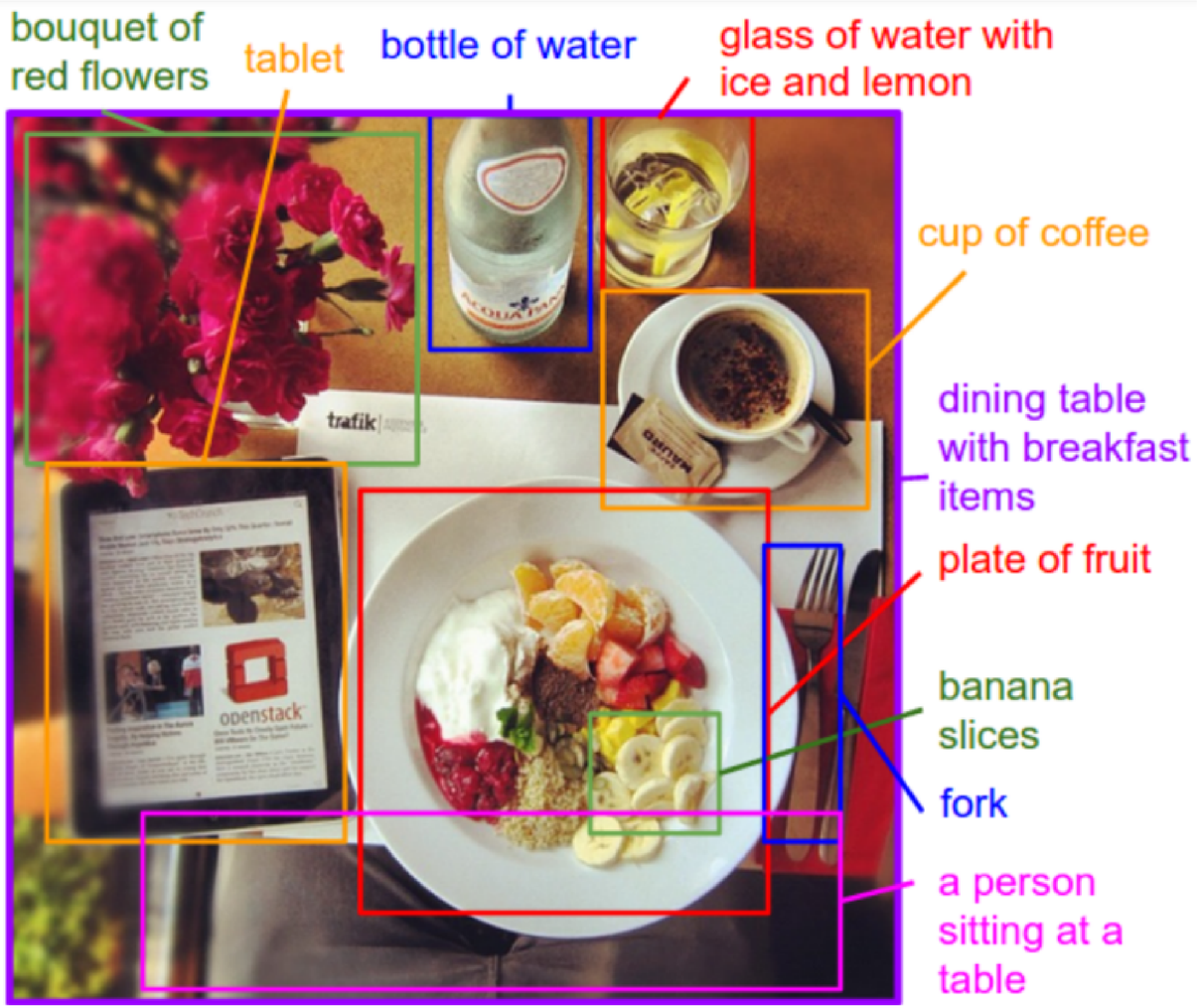


Figure 6: Motivation figure: the model treats language as a rich label space and generates descriptions of image regions.

8.3 Deep Visual-Semantic Alignments

- the ultimate goal of the model is to generate descriptions of image regions
- during training, the input to the model is a set of images and their corresponding sentence descriptions (Figure 7)
- the approach first introduces a model that aligns sentence snippets to the visual regions that they describe through a multimodal embedding
- it then treats these correspondences as training data for a second, multimodal RNN model that learns to generate the snippets
- the alignment model assumes an input dataset of images and their sentence descriptions
- the key insight is that sentences written by people make frequent references to some particular, but unknown location in the image
- for example, in Figure 7, the words “Tabby cat is leaning” refer to the cat, the words “wooden table” refer to the table, etc.
- we would like to infer these latent correspondences, with the eventual goal of later learning to generate these snippets from image regions
- first, neural networks that map words and image regions into a common, multimodal embedding, are introduced
- then a novel objective is introduced, which learns the embedding representations so that semantically similar concepts across the two modalities occupy nearby regions of the space

8.3 Deep Visual-Semantic Alignments

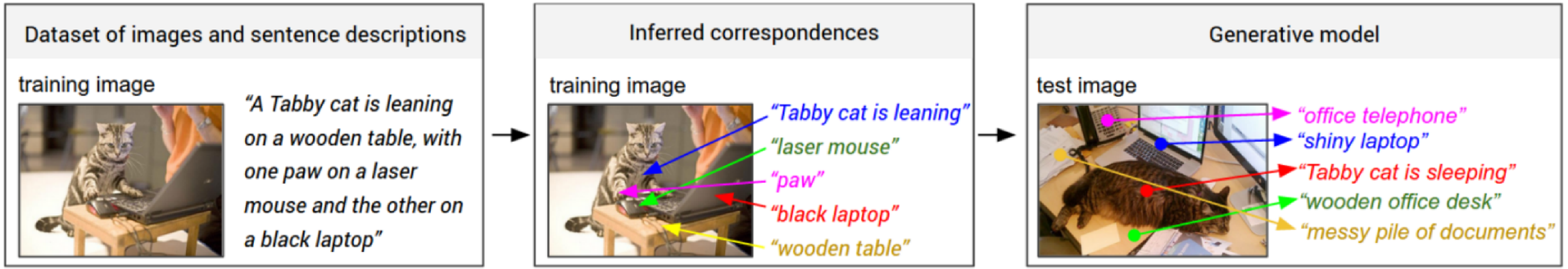


Figure 7: Overview of the approach. A dataset of images and their sentence descriptions is the input to the model (left). The model first infers the correspondences (middle) and then learns to generate novel descriptions (right).

8.3 Deep Visual-Semantic Alignments

- it can be observed that sentence descriptions make frequent references to objects and their attributes
- thus, objects in every image with a Region Convolutional Neural Network (RCNN) are detected
- the CNN is pre-trained on ImageNet
- the top 19 detected locations are used in addition to the whole image and the representations based on the pixels I_b inside each bounding box are computed as follows:

$$v = W_m[CNN_{\theta_c}(I_b)] + b_m,$$

  where $CNN(I_b)$ transforms the pixels inside bounding box I_b into 4096-dimensional activations of the fully-connected layer immediately before the classifier
- the CNN parameters θ_c contain approximately 60 million parameters
- the matrix W_m has dimensions $h \times 4096$, where h is the size of the multimodal embedding space (h ranges from 1000-1600)
- every image is thus represented as a set of h-dimensional vectors $\{v_i | i = 1 \ldots 20\}$

8.3 Deep Visual-Semantic Alignments

- to establish the inter-modal relationships, we would like to represent the words in the sentence in the same *h*-dimensional embedding space that the image regions occupy
- the simplest approach might be to project every individual word directly into this embedding
- however, this approach does not consider any ordering and word context information in the sentence
- to address these concerns, a Bidirectional Recurrent Neural Network (BRNN) is used to compute the word representations
- the BRNN takes a sequence of *N* words (encoded in a 1-of-k representation) and transforms each one into an *h*-dimensional vector
- however, the representation of each word is enriched by a variably-sized context around that word

8.3 Deep Visual-Semantic Alignments

- using the index $t = 1 \ldots N$ to denote the position of a word in a sentence, the precise form of the BRNN is as follows:

$$\begin{aligned} x_t &= W_w \mathbb{I}_t \\ e_t &= f(W_e x_t + b_e) \\ h_t^f &= f(e_t + W_f h_{t-1}^f + b_f) \\ h_t^b &= f(e_t + W_b h_{t+1}^b + b_b) \\ s_t &= f(W_d(h_t^f + h_t^b) + b_d). \end{aligned}$$

- here, $\mathbb{I}_t$ is an indicator column vector that has a single one at the index of the t-th word in a word vocabulary
- the weights W_w specify a 300-dimensional word embedding matrix
- note that the BRNN consists of two independent streams of processing, one moving left to right (h_t^f) and the other right to left (h_t^b) (see Figure 8 for diagram)
- the final h-dimensional representation s_t for the t-th word is a function of both the word at that location and also its surrounding context in the sentence

8.3 Deep Visual-Semantic Alignments

- technically, every s_t is a function of all words in the entire sentence, but the final word representations (s_t) align most strongly to the visual concept of the word at that location ($\mathbb{I}_t$)
- the parameters W_e, W_f, W_b, W_d and the respective biases b_e, b_f, b_b, b_d are learned
- a typical size of the hidden representation ranges between 300-600 dimensions
- the activation function f is ReLU
- we have described the transformations that map every image and sentence into a set of vectors in a common h-dimensional space
- since the supervision is at the level of entire images and sentences, the strategy is to formulate an image-sentence score as a function of the individual region-word scores
- intuitively, a sentence-image pair should have a high matching score if its words have a confident support in the image

8.3 Deep Visual-Semantic Alignments

- the dot product $v_i^\top s_t$ between the i-th region and t-th word can be interpreted as a measure of similarity and we can use it to define the score between image k and sentence l as:

$$S_{kl} = \sum_{t \in g_l} \sum_{i \in g_k} \max(0, v_i^\top s_t).$$

- here, g_k is the set of image fragments in image k and g_l is the set of sentence fragments in sentence l
- the indices k, l range over the images and sentences in the training set
- this score carries the interpretation that a sentence fragment aligns to a subset of the image regions whenever the dot product is positive
- the following reformulation simplifies the model and alleviates the need for additional objectives and their hyperparameters:

$$S_{kl} = \sum_{t \in g_l} \max_{i \in g_k} v_i^\top s_t.$$

- here, every word s_t aligns to the single best image region

8.3 Deep Visual-Semantic Alignments

- assuming that $k = l$ denotes a corresponding image and sentence pair, the final max-margin, structured loss remains:

$$\mathcal{C}(\theta) = \sum_k \left[\underbrace{\sum_l \max(0, S_{kl} - S_{kk} + 1)}_{\text{rank images}} + \underbrace{\sum_l \max(0, S_{lk} - S_{kk} + 1)}_{\text{rank sentences}} \right].$$

- this objective encourages aligned image-sentences pairs to have a higher score than misaligned pairs, by a margin
- now, consider an image from the training set and its corresponding sentence
- we can interpret the quantity $v_i^\top s_t$ as the unnormalized log probability of the t-th word describing any of the bounding boxes in the image
- however, since we are ultimately interested in generating snippets of text instead of single words, we would like to align extended, contiguous sequences of words to a single bounding box

8.3 Deep Visual-Semantic Alignments

- note that the naive solution that assigns each word independently to the highest-scoring region is insufficient because it leads to words getting scattered inconsistently to different regions
- to address this issue, the true alignments are treated as latent variables in a Markov Random Field (MRF), where the binary interactions between neighboring words encourage an alignment to the same region
- concretely, given a sentence with N words and an image with M bounding boxes, we introduce the latent alignment variables $a_j \in \{1 \dots M\}$ for $j = 1 \dots N$ and formulate an MRF in a chain structure along the sentence as follows:

$$E(\boldsymbol{a}) = \sum_{j=1 \dots N} \psi_j^U(a_j) + \sum_{j=1 \dots N-1} \psi_j^B(a_j, a_{j+1}),$$

$$\psi_j^U(a_j = t) = v_i^\top s_t,$$

$$\psi_j^B(a_j, a_{j+1}) = \beta \mathbf{1}_{a_j = a_{j+1}}.$$

8.3 Deep Visual-Semantic Alignments

- here, β is a hyperparameter that controls the affinity towards longer word phrases
- this parameter allows us to interpolate between single-word alignments ($\beta = 0$) and aligning the entire sentence to a single, maximally scoring region when β is large
- the energy is minimized to find the best alignments $\boldsymbol{a}$ using dynamic programming
- the output of this process is a set of image regions annotated with segments of text
- we now describe an approach for generating novel phrases based on these correspondences

8.3 Deep Visual-Semantic Alignments

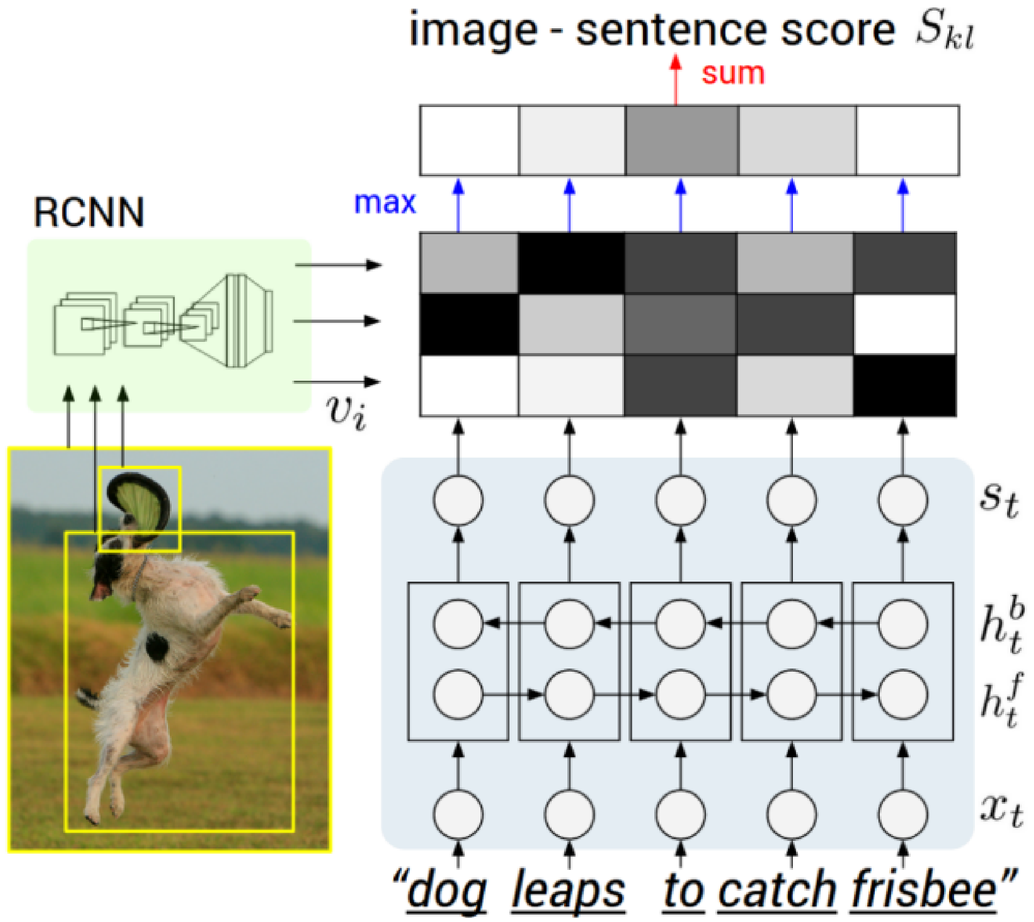


Figure 8: Diagram for evaluating the image-sentence score S_{kl}. Object regions are embedded with a CNN (left). Words (enriched by their context) are embedded in the same multimodal space with a BRNN (right). Pairwise similarities are computed with inner products (magnitudes shown in grayscale) and finally reduced to image-sentence score.

8.3 Deep Visual-Semantic Alignments

- now, we assume an input set of images and their textual descriptions
- these could be full images and their sentence descriptions, or regions and text snippets, as inferred previously
- the key challenge is in the design of a model that can predict a variable-sized sequence of outputs given an image
- in language models based on RNNs, this is achieved by defining a probability distribution of the next word in a sequence given the current word and context from previous time steps
- a simple but effective extension that additionally conditions the generative process on the content of an input image is introduced
- more formally, during training, the multimodal RNN takes the image pixels I and a sequence of input vectors $(x_1, \ldots, x_T)$

8.3 Deep Visual-Semantic Alignments

- it then computes a sequence of hidden states $(h_1, \ldots, h_t)$ and a sequence of outputs $(y_1, \ldots, y_t)$ by iterating the following recurrence relation for $t = 1$ to T:

$$\begin{aligned} b_v &= W_{hi}[CNN_{\theta_c}(I)], \\ h_t &= f(W_{hx}x_t + W_{hh}h_{t-1} + b_h + \mathbf{1}_{t=1} \odot b_v), \\ y_t &= \text{softmax}(W_{oh}h_t + b_o). \end{aligned}$$

- in the equations above, W_{hi}, W_{hx}, W_{hh}, W_{oh}, and b_h, b_o are learnable parameters, and $CNN_{\theta_c}(I)$ is the last layer of a CNN
- the output vector y_t holds the (unnormalized) log probabilities of words in the dictionary and one additional dimension for a special END token
- note that we provide the image context vector b_v to the RNN only at the first iteration
- a typical size of the hidden layer of the RNN is 512 neurons
- the RNN is trained to combine a word (x_t), the previous context (h_{t-1}) to predict the next word (y_t)
- we condition the RNN's predictions on the image information (b_v) via bias interactions on the first step

8.3 Deep Visual-Semantic Alignments

- the training proceeds as follows (see Figure 9): we set $h_0 = \mathbf{0}$, x_1 to a special START vector, and the desired label y_1 as the first word in the sequence
- analogously, we set x_2 to the word vector of the first word and expect the network to predict the second word, etc.
- finally, on the last step, when x_T represents the last word, the target label is set to a special END token
- the cost function is to maximize the log probability assigned to the target labels (i.e., softmax classifier)
- to predict a sentence, we compute the image representation b_v, set $h_0 = \mathbf{0}$, x_1 to the START vector and compute the distribution over the first word y_1
- we sample a word from the distribution (or pick the argmax), set its embedding vector as x_2, and repeat this process until the END token is generated

8.3 Deep Visual-Semantic Alignments

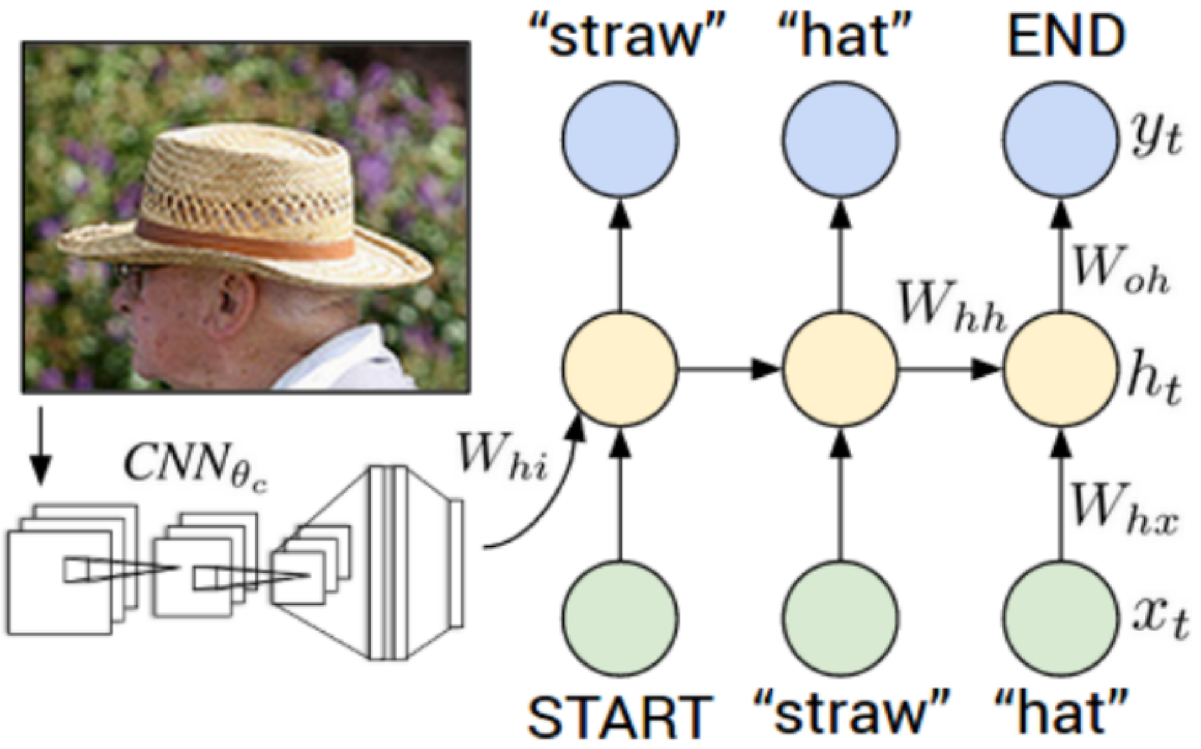


Figure 9: Diagram of the multimodal RNN generative model. The RNN takes a word, the context from previous time steps and defines a distribution over the next word in the sentence. The RNN is conditioned on the image information at the first time step. START and END are special tokens.

8.4 Visual Question Answering

- a VQA dataset containing $\sim$0.25M images, $\sim$0.76M questions, and $\sim$10M answers is constructed
- in order to perform VQA, a two-channel vision (image) + language (question) model is developed that culminates with a softmax over K possible outputs
- the top K = 1000 most frequent answers are chosen as possible outputs
- this set of answers covers 82.67% of the train+validation answers in the VQA dataset
- the different components of the model are described below
- the **Image Channel** provides an embedding for the image
- two types of embeddings are used:
  1. **I**: The activations from the last hidden layer of the VGG network are used as 4096-dimensional image embedding.
  2. **norm I**: These are ℓ_2-normalized activations from the last hidden layer of the VGG network.

8.4 Visual Question Answering

- the **Question Channel** provides an embedding for the question
- two types of embeddings are used:
  1. **LSTM Q**: An LSTM with one hidden layer is used to obtain a 1024-dimensional embedding for the question. The embedding obtained from the LSTM is a concatenation of last cell state and last hidden state representations (each being 512-dimensional) from the hidden layer of the LSTM. Each question word is encoded with a 300-dimensional embedding by a fully-connected layer + tanh nonlinearity which is then fed to the LSTM. The input vocabulary to the embedding layer consists of all the question words seen in the training dataset.
  2. **deeper LSTM Q**: An LSTM with two hidden layers is used to obtain a 2048-dimensional embedding for the question. The embedding obtained from the LSTM is a concatenation of the last cell state and last hidden state representations (each being 512-dimensional) from each of the two hidden layers of the LSTM. Hence the 2 (hidden layers) $\times$ 2 (cell state and hidden state) $\times$ 512 (dimensionality of each of the cell states, as well as hidden states) in Figure 10. This is followed by a fully-connected layer + tanh nonlinearity to transform the 2048-dimensional embedding to 1024 dimensions. The question words are encoded in the same way as in LSTM Q.

8.4 Visual Question Answering

- then, the image and question embeddings are combined to obtain a single embedding by the **Multi-Layer Perceptron (MLP)**
- for both the LSTM Q + I and deeper LSTM Q + norm I (Figure 10) methods, the image embedding is first transformed to 1024-dimensional by a fully-connected layer + tanh nonlinearity to match the LSTM embedding of the question
- the transformed image and LSTM embeddings (being in a common space) are then fused via element-wise multiplication
- this combined image + question embedding is then passed to an MLP – a fully-connected neural network classifier with 2 hidden layers and 1000 hidden units (dropout 0.5) in each layer with tanh nonlinearity, followed by a softmax layer to obtain a distribution over K answers
- the entire model is learned end-to-end with a cross-entropy loss
- the VGG network parameters are frozen to those learned for ImageNet classification and not fine-tuned in the image channel

8.4 Visual Question Answering

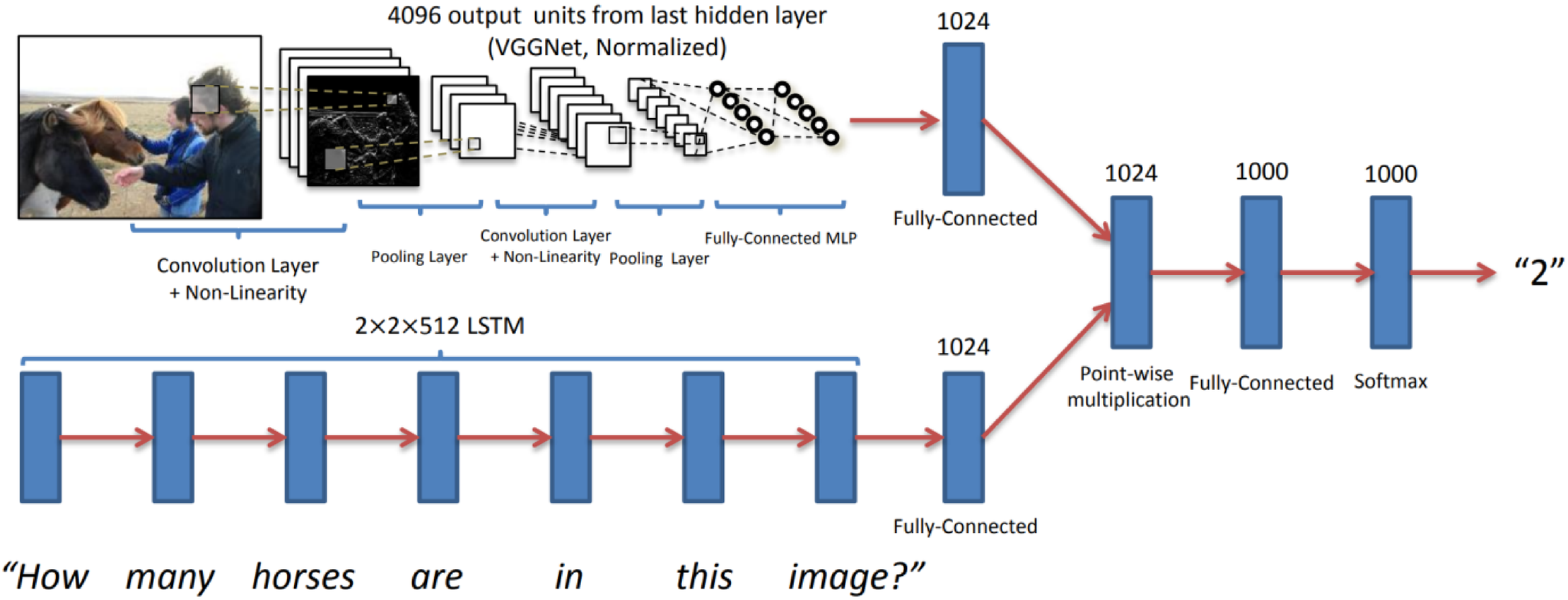


Figure 10: The best performing model (deeper LSTM Q + norm I). This model uses a two-layer LSTM to encode the questions and the last hidden layer of a VGG network to encode the images. The image features are then ℓ_2-normalized. Both the question and image features are transformed to a common space and fused via element-wise multiplication, which is then passed through a fully-connected layer followed by a softmax layer to obtain a distribution over answers.

8.5 Visual Dialog

- a visual dialog (VisDial) dataset containing 10 question-answer pairs on $\sim$120k images, with a total of $\sim$1.2M dialog question-answer pairs, is constructed
- also, a family of neural encoder-decoder models for visual dialog with 3 encoders – Late Fusion, Hierarchical Recurrent Encoder, and Memory Network – and 2 decoders (generative and discriminative) are introduced
- a number of neural visual dialog answerer models are developed
- the model is given as input – an image I, the "ground-truth" dialog history (including the image caption) $H = (\underbrace{C}_{H_0}, \underbrace{(Q_1, A_1)}_{H_1}, \ldots, \underbrace{(Q_{t-1}, A_{t-1})}_{H_{t-1}})$, the question Q_t, and a list of 100 candidate answers $\mathcal{A}_t = \{A_t^{(1)}, \ldots, A_t^{(100)}\}$ – and asked to return a sorting of $\mathcal{A}_t$
- at a high level, all the models follow the encoder-decoder framework, i.e., are factorized into two parts – (1) an encoder that converts the input (I, H, Q_t) into a vector space, and (2) a decoder that converts the embedded vector into an output
- we describe choices for each component next

8.5 Visual Dialog

- **Decoders**: two types of decoders are used:
  - **Generative** (LSTM) decoder: where the encoded vector is set as the initial state of the LSTM RNN language model. During training, the log-likelihood of the ground truth answer sequence is maximized given its corresponding encoded representation (trained end-to-end). To evaluate, the model's log-likelihood scores are used and candidate answers are ranked. Note that this decoder does not need to score options during training. As a result, such models do not exploit the biases in option creation and typically underperform models that do, but it is debatable whether exploiting such biases is really indicative of progress. Moreover, generative decoders are more practical in that they can actually be deployed in realistic applications.
  - **Discriminative** (softmax) decoder: computes dot product similarity between input encoding and an LSTM encoding of each of the answer options. These dot products are fed into a softmax to compute the posterior probability over options. During training, the log-likelihood of the correct option is maximized. During evaluation, options are simply ranked based on their posterior probabilities.

8.5 Visual Dialog

- **Encoders**: three different encoders (listed below) that convert inputs (I, H, Q_t) into a joint representation are developed
- in all cases, I is represented via the ℓ_2-normalized activations from the penultimate layer of VGG-16
- for each encoder E, all possible versions are experimented with: $E(Q_t)$, $E(Q_t, I)$, $E(Q_t, H)$, $E(Q_t, I, H)$ (for some encoders, not all combinations are "valid"; details below)
  - **Late Fusion (LF) Encoder**: In this encoder, we treat H as a long string with the entire history $(H_0, \ldots, H_{t-1})$ concatenated. Q_t and H are separately encoded with two different LSTMs, and individual representations of participating inputs (I, H, Q_t) are concatenated and linearly transformed to a desired size of joint representation.

8.5 Visual Dialog

- **Hierarchical Recurrent Encoder (HRE)**: In this encoder, the intuition that there is a hierarchical nature to the problem is captured – each question Q_t is a sequence of words that need to be embedded, and the dialog as a whole is a sequence of question-answer pairs (Q_t, A_t). Thus, as shown in Figure 11, an HRE model is proposed that contains a dialog-RNN sitting on top of a recurrent block (R_t). The recurrent block R_t embeds the question and image jointly via an LSTM (early fusion), embeds each round of the history H_t, and passes a concatenation of these to the dialog-RNN above it. The dialog-RNN produces both an encoding for this round (E_t in Figure 11) and a dialog context to pass onto the next round. We also add an attention-over-history ("Attention" in Figure 11) mechanism allowing the recurrent block R_t to choose and attend to the round of the history relevant to the current question. This attention mechanism consists of a softmax over previous rounds $(0, 1, \ldots, t-1)$ computed from the history and question+image encoding.

8.5 Visual Dialog

-
  - **Memory Network (MN) Encoder**: A MN encoder is developed that maintains each previous question and answer as a "fact" in its memory bank and learns to refer to the stored facts and image to answer the question. Specifically, Q_t is encoded with an LSTM to get a 512-dimensional vector, each previous round of history $(H_0, \ldots, H_{t-1})$ is encoded with another LSTM to get a $t \times 512$ matrix. We compute inner product of question vector with each history vector to get scores over previous rounds, which are fed to a softmax to get attention-over-history probabilities. Convex combination of history vectors using these attention probabilities gives us the "context vector", which is passed through a fully-connected layer and added to the question vector to construct the MN encoding.
- we use a "[encoder]-[input]-[decoder]" convention to refer to model-input combinations
- for example, "LF-QI-D" has a Late Fusion encoder with question+image inputs (no history), and a discriminative decoder
- implementation details about the models are given next

8.5 Visual Dialog

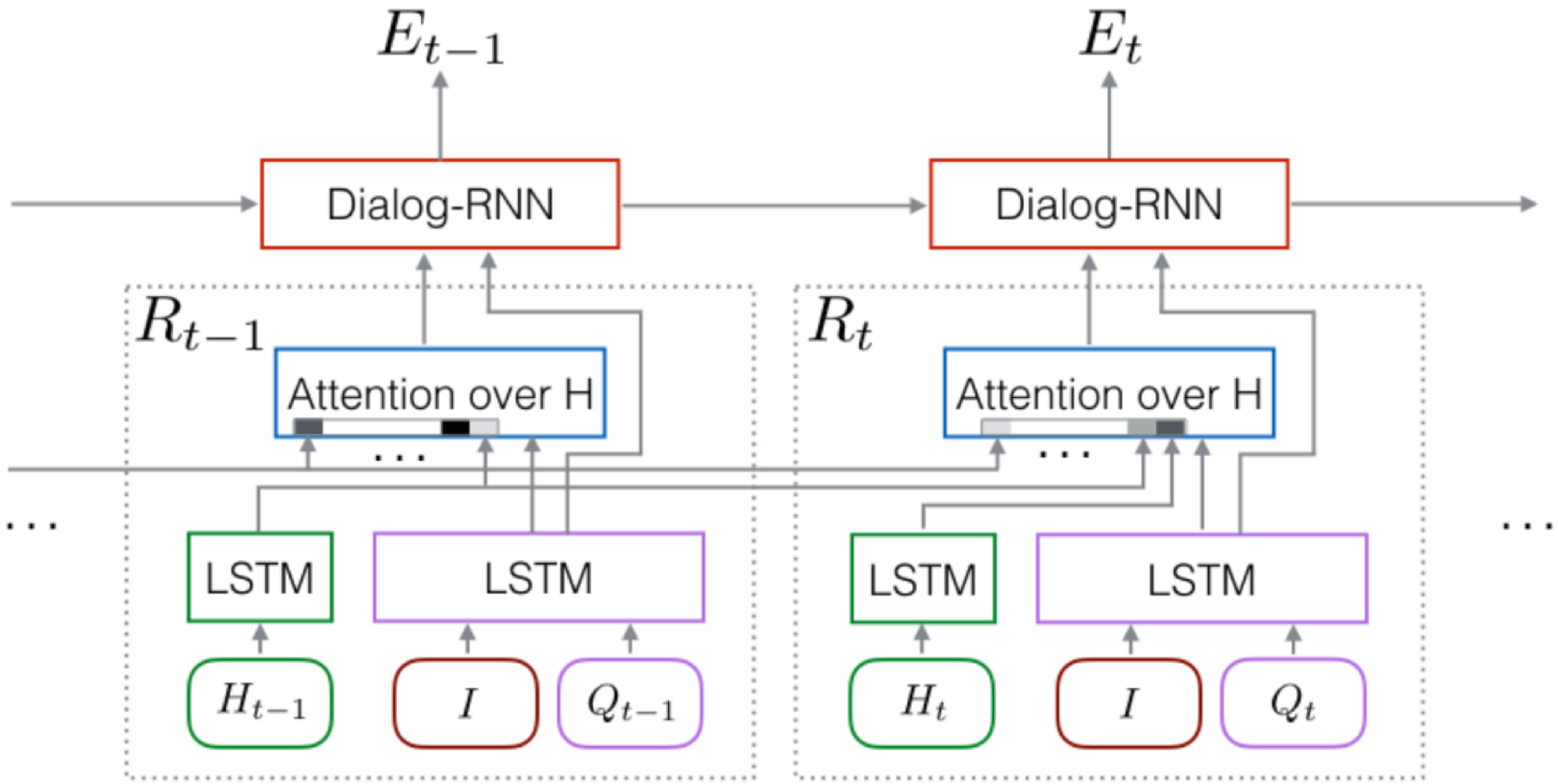


Figure 11: Architecture of Hierarchical Recurrent Encoder (HRE) encoder with attention. At the current round R_t, the model has the capability to choose and attend to relevant history from previous rounds, based on the current question. This attention-over-history feeds into a dialog-RNN along with the question to generate joint representation E_t for the decoder.

8.5 Visual Dialog

- **Late Fusion (LF) Encoder**: the image is encoded with a VGG-16 CNN, the question and concatenated history are encoded with separate LSTMs, and the three representations are concatenated
- this is followed by a fully-connected layer and tanh nonlinearity to a 512-dimensional vector, which is used to decode the response
- Figure 12 shows the model architecture for the LF encoder

8.5 Visual Dialog

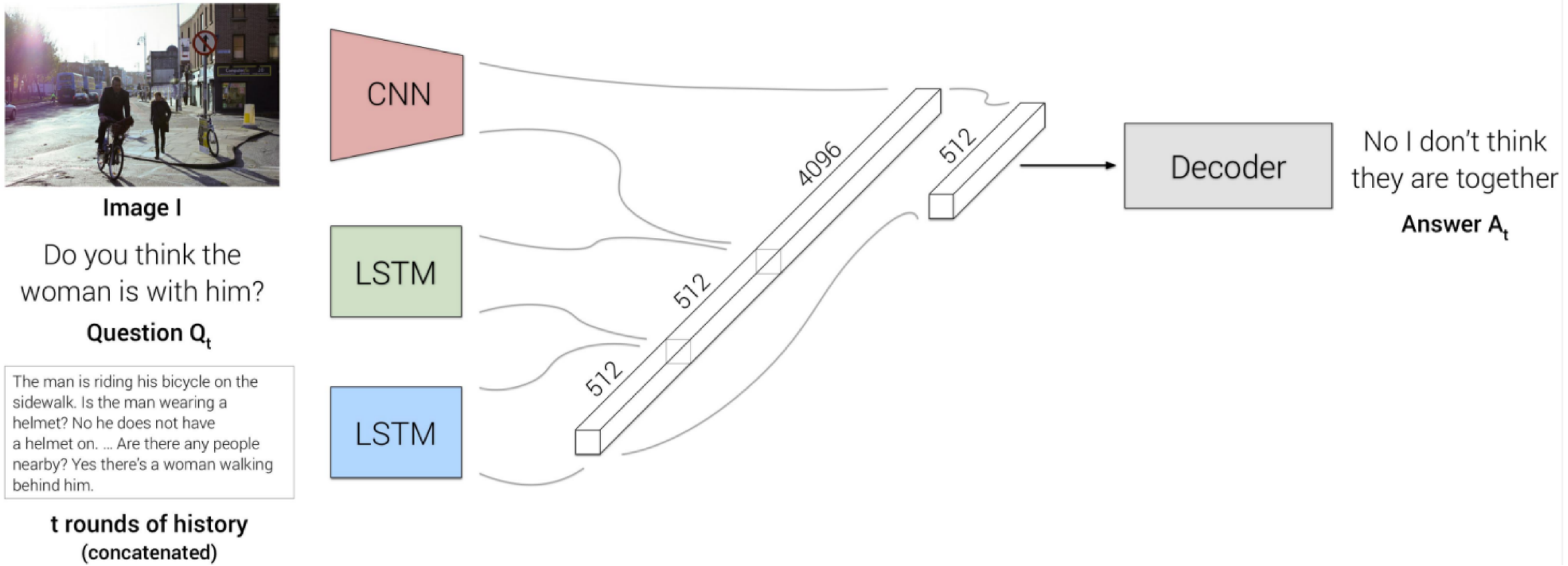


Figure 12: Late Fusion (LF) Encoder.

8.5 Visual Dialog

- **Hierarchical Recurrent Encoder (HRE)**: in this encoder, the image representation from VGG-16 CNN is early fused with the question
- specifically, the image representation is concatenated with every question word as it is fed to an LSTM
- each QA-pair in dialog history is independently encoded by another LSTM with shared weights
- the image-question representation, computed for every round from 1 through t, is concatenated with history representation from the previous round and constitutes a sequence of question-history vectors
- these vectors are fed as input to a dialog-level LSTM, whose output state at t is used to decode the response to Q_t
- Figure 13 shows the model architecture for the HRE

8.5 Visual Dialog

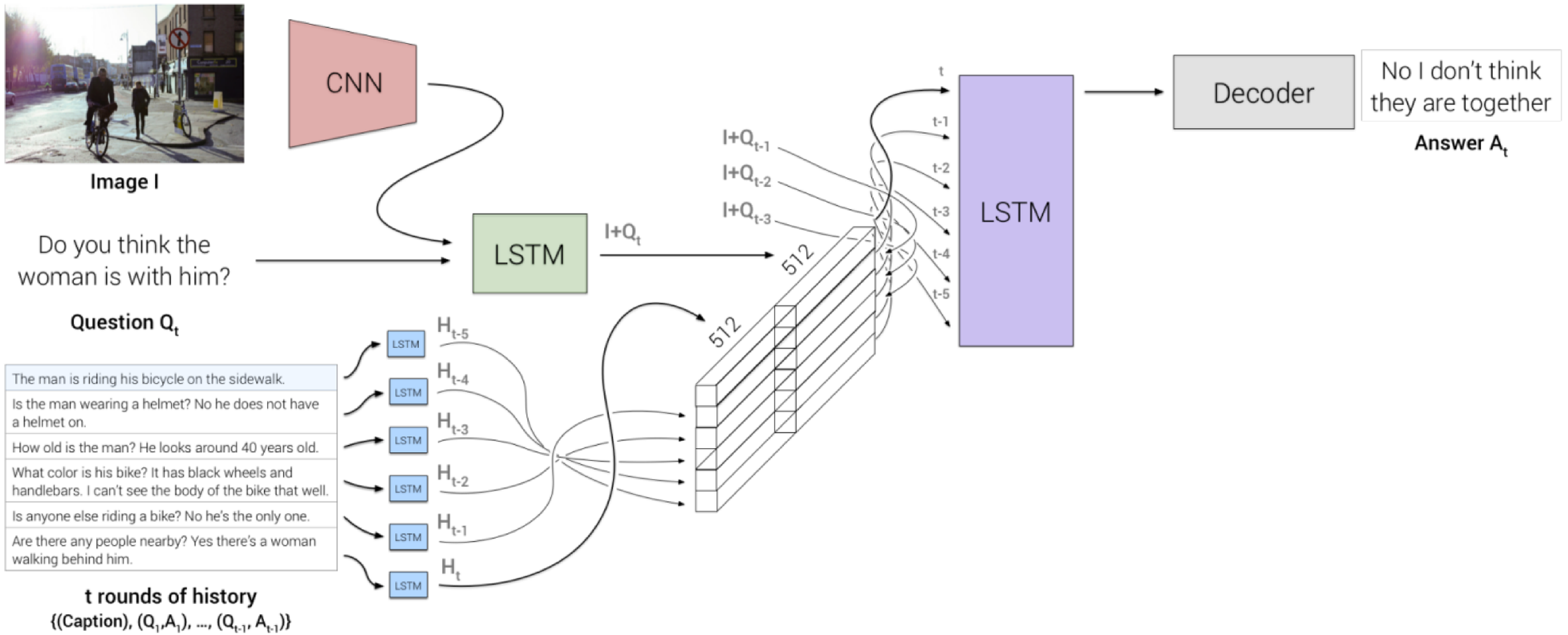


Figure 13: Hierarchical Recurrent Encoder (HRE).

8.5 Visual Dialog

- **Memory Network**: the image is encoded with a VGG-16 CNN and question with an LSTM
- the representations are concatenated and followed by a fully-connected layer and tanh nonlinearity to get a “query vector”
- each caption/QA-pair (or “fact”) in dialog history is encoded independently by an LSTM with shared weights
- the query vector is then used to compute attention over the t facts by inner product
- convex combination of attended history vectors is passed through a fully-connected layer and tanh nonlinearity, and added back to the query vector
- this combined representation is then passed through another fully-connected layer and tanh nonlinearity and then used to decode the response
- the model architecture is shown in Figure 14

8.5 Visual Dialog

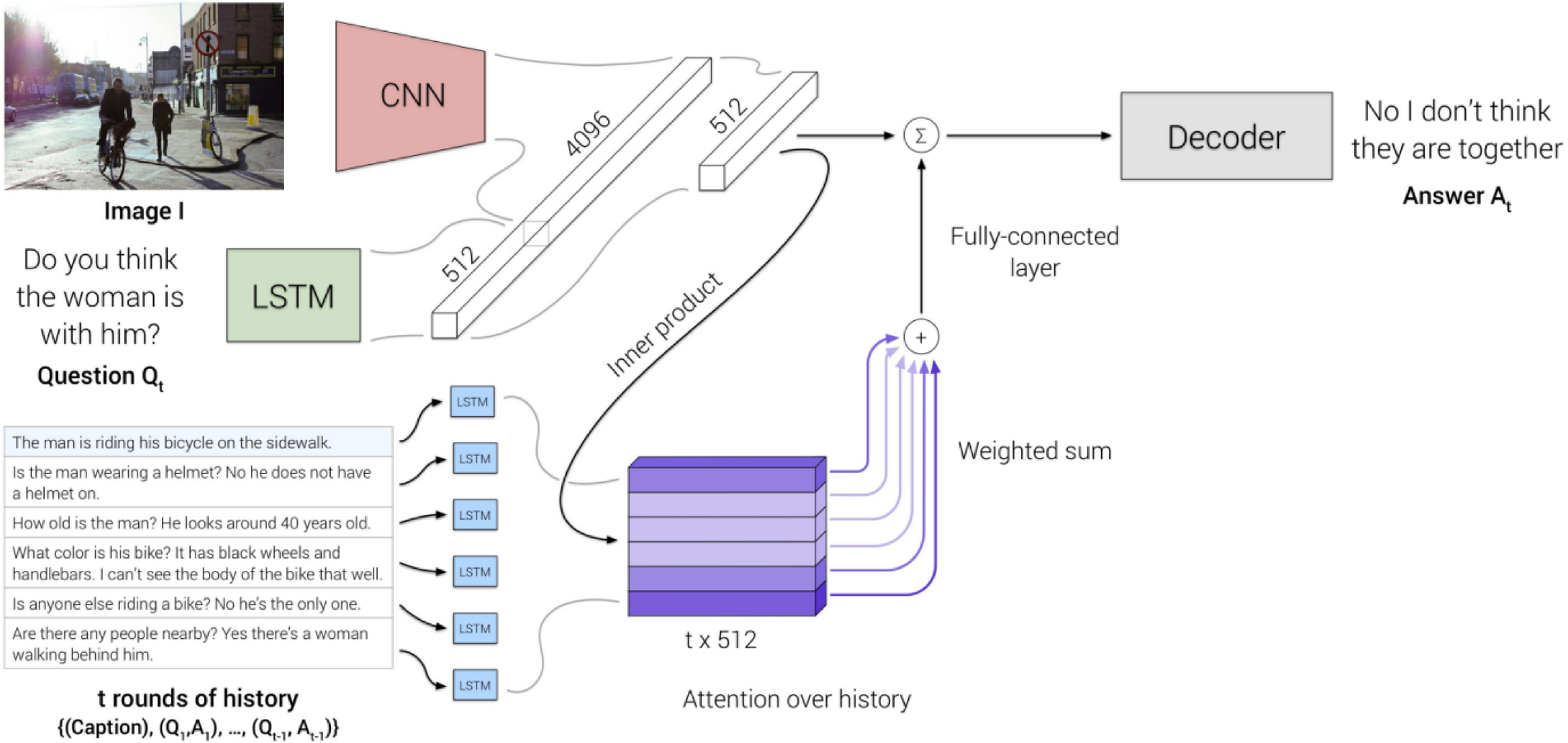


Figure 14: Memory Network Encoder (MNE).

Vă mulţumesc!